

AI Assisted Coding Assignment - 4

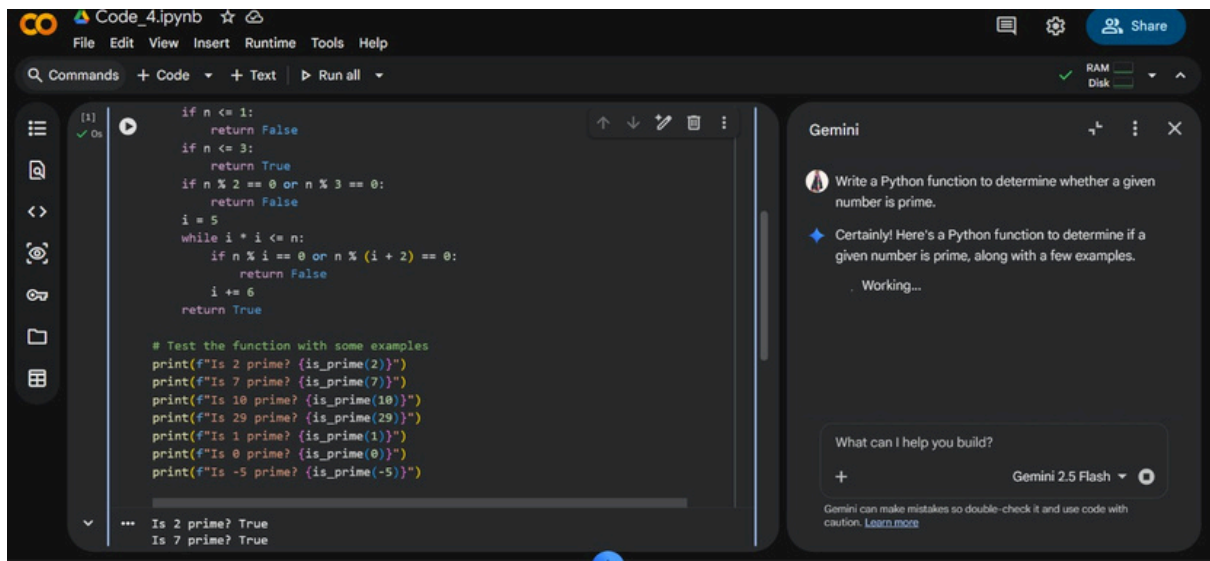
Chilukala Sai Architha(2303A52220) - Batch 37

Task 1 – Zero-Shot Prompting

Prompt Used

"Write a Python function to determine whether a given number is prime."

(No examples were provided.)



The screenshot shows a Jupyter Notebook environment. The left pane displays a Python script for a prime-checking function. The right pane shows a Gemini AI chat window with the prompt and the AI's response.

```
[1] if n <= 1:
    return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

# Test the function with some examples
print(f"Is 2 prime? {is_prime(2)}")
print(f"Is 7 prime? {is_prime(7)}")
print(f"Is 10 prime? {is_prime(10)}")
print(f"Is 29 prime? {is_prime(29)}")
print(f"Is 1 prime? {is_prime(1)}")
print(f"Is 0 prime? {is_prime(0)}")
print(f"Is -5 prime? {is_prime(-5)}")
```

Output: Is 2 prime? True, Is 7 prime? True

Gemini chat window:

Write a Python function to determine whether a given number is prime.

Certainly! Here's a Python function to determine if a given number is prime, along with a few examples.

Working...

What can I help you build?

Gemini 2.5 Flash

Explanation

- Numbers less than or equal to 1 are not prime.
- Loop checks divisibility from 2 to $\sqrt{n}$.
- If divisible, return False; otherwise return True.

Output

True

False

Observation

The AI correctly generated a prime-checking function using only instructions without examples.

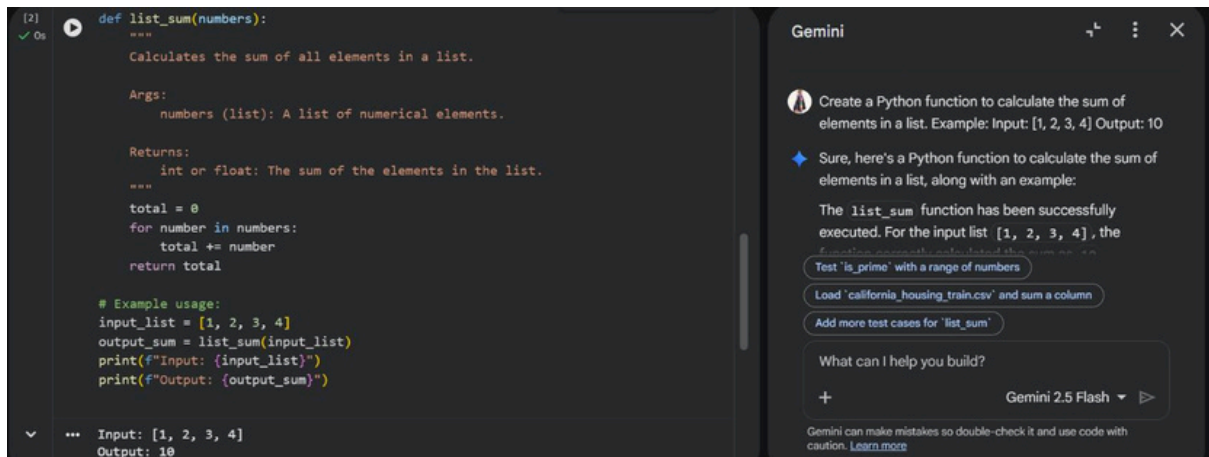
Task 2 – One-Shot Prompting

Prompt Used

"Create a Python function to calculate the sum of elements in a list. Example:

Input: [1, 2, 3, 4]

Output: 10"



The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor displays a Python function `list_sum` that calculates the sum of elements in a list. The function includes docstrings for its purpose, arguments, and return values. An example usage is provided at the bottom of the code block. The Gemini chat interface on the right shows the user's prompt and the model's response, which includes the function code and an example output.

```
[2] def list_sum(numbers):  
    """  
    Calculates the sum of all elements in a list.  
  
    Args:  
        numbers (list): A list of numerical elements.  
  
    Returns:  
        int or float: The sum of the elements in the list.  
    """  
    total = 0  
    for number in numbers:  
        total += number  
    return total  
  
# Example usage:  
input_list = [1, 2, 3, 4]  
output_sum = list_sum(input_list)  
print(f"Input: {input_list}")  
print(f"Output: {output_sum}")
```

Input: [1, 2, 3, 4]
Output: 10

Gemini

Create a Python function to calculate the sum of elements in a list. Example: Input: [1, 2, 3, 4] Output: 10

Sure, here's a Python function to calculate the sum of elements in a list, along with an example:

The `list_sum` function has been successfully executed. For the input list `[1, 2, 3, 4]`, the

Test 'is_prime' with a range of numbers

Load 'california_housing_train.csv' and sum a column

Add more test cases for 'list_sum'

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Explanation

- Initializes total to 0.
- Adds each number from the list to total.
- Returns the final sum.

Output

10

Observation

The single example helped the AI clearly understand what operation was required.

Task 3 – Few-Shot Prompting

Prompt Used

"Create a function to extract digits from an alphanumeric string.

Examples:

Input: 'a1b2c3' → Output: '123'

Input: 'x9y8z' → Output: '98'

Input: 'abc123' → Output: '123'"

```
Args:
  alphanumeric_string (str): The input string containing alphanumeric characters.

Returns:
  str: A new string containing only the digits from the input string.

"""
digits = ''
for char in alphanumeric_string:
    if char.isdigit():
        digits += char
return digits

# Example usage:
print(f"Input: 'a1b2c3' -> Output: '{extract_digits('a1b2c3')}')")
print(f"Input: 'x9y8z' -> Output: '{extract_digits('x9y8z')}')")
print(f"Input: 'abc123' -> Output: '{extract_digits('abc123')}')")
print(f"Input: 'no_digits' -> Output: '{extract_digits('no_digits')}')")
print(f"Input: '12345' -> Output: '{extract_digits('12345')}')")
```

Input: 'a1b2c3' -> Output: '123'

Input: 'x9y8z' -> Output: '98'

Input: 'abc123' -> Output: '123'

Input: 'no_digits' -> Output: ''

Input: '12345' -> Output: '12345'

Gemini

Create a function to extract digits from an alphanumeric string. Examples: Input: 'a1b2c3' -> Output: '123' Input: 'x9y8z' -> Output: '98' Input: 'abc123' -> Output: '123'

Certainly! Here's a Python function that extracts digits from an alphanumeric string, along with the examples

Explain the 'is_prime' function in detail

Combine 'extract_digits' and 'list_sum' to sum digits from a list of strings

How can I add more tests for the 'extract_digits' function

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Explanation

- Loops through each character in the string.
- Checks if the character is a digit using `isdigit()`.
- Appends digits to a result string.

Output

123

98

123

Observation

Multiple examples improved accuracy and clarity of logic.

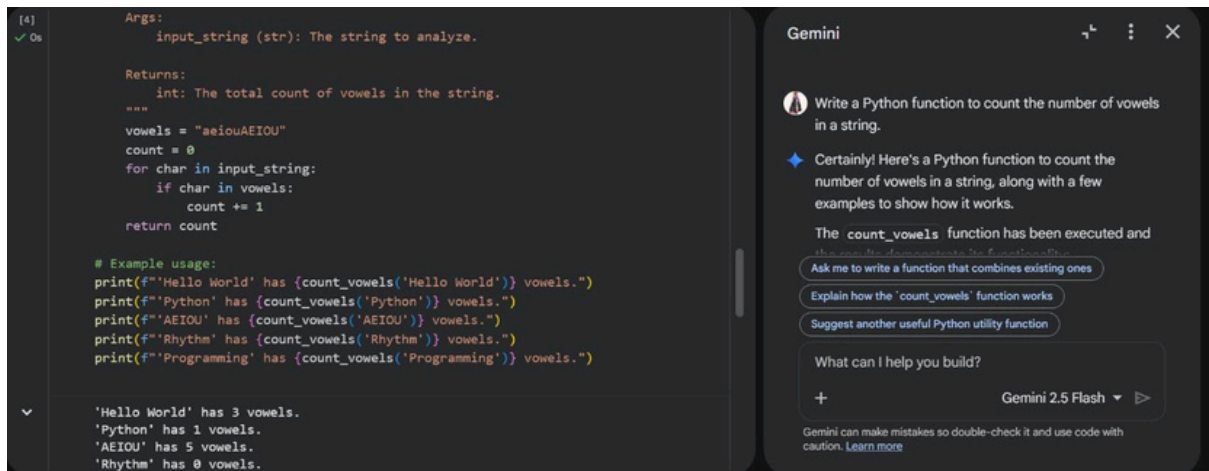
Task 4 – Zero-Shot vs Few-Shot Comparison

Problem

Generate a function to count the number of vowels in a string.

Zero-Shot Prompt

"Write a Python function to count the number of vowels in a string."



Output

3

Few-Shot Prompt

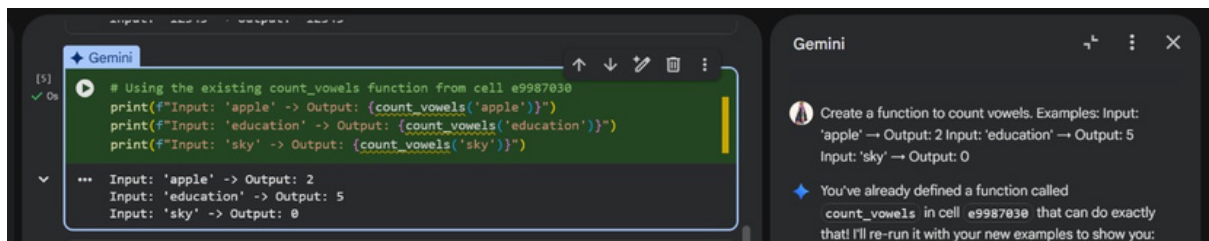
"Create a function to count vowels.

Examples:

Input: 'apple' → Output: 2

Input: 'education' → Output: 5

Input: 'sky' → Output: 0"



Output 2 5 0 Comparison & Explanation

Aspect	Zero-Shot	Few-Shot
Clarity	Basic logic	More optimized
Accuracy	Correct	Correct

Task 5 – Few-Shot Prompting

Prompt Used

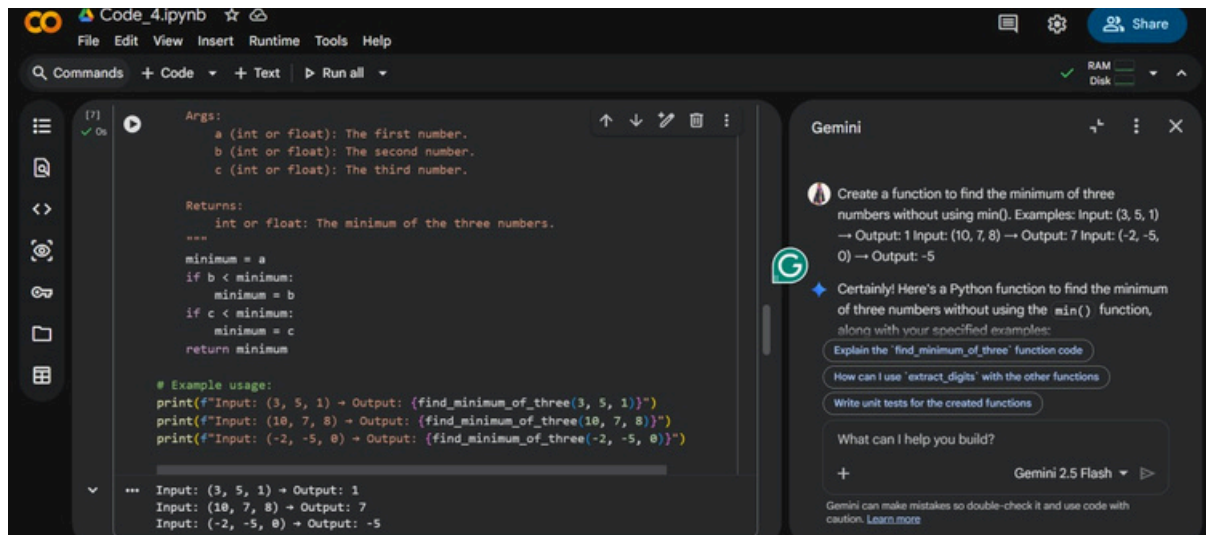
"Create a function to find the minimum of three numbers without using min().

Examples:

Input: (3, 5, 1) → Output: 1

Input: (10, 7, 8) → Output: 7

Input: (-2, -5, 0) → Output: -5"



```
def find_minimum_of_three(a, b, c):  
    """  
    a (int or float): The first number.  
    b (int or float): The second number.  
    c (int or float): The third number.  
    Returns:  
    int or float: The minimum of the three numbers.  
    """  
    minimum = a  
    if b < minimum:  
        minimum = b  
    if c < minimum:  
        minimum = c  
    return minimum  
  
# Example usage:  
print(f"Input: (3, 5, 1) → Output: {find_minimum_of_three(3, 5, 1)}")  
print(f"Input: (10, 7, 8) → Output: {find_minimum_of_three(10, 7, 8)}")  
print(f"Input: (-2, -5, 0) → Output: {find_minimum_of_three(-2, -5, 0)}")
```

Output:

```
Input: (3, 5, 1) → Output: 1  
Input: (10, 7, 8) → Output: 7  
Input: (-2, -5, 0) → Output: -5
```

Gemini

Create a function to find the minimum of three numbers without using min(). Examples: Input: (3, 5, 1) → Output: 1 Input: (10, 7, 8) → Output: 7 Input: (-2, -5, 0) → Output: -5

Certainly! Here's a Python function to find the minimum of three numbers without using the `min()` function, along with your specified examples:

Explain the 'find_minimum_of_three' function code

How can I use 'extract_digits' with the other functions

Write unit tests for the created functions

What can I help you build?

+ Gemini 2.5 Flash

Explanation

- Assume first number is smallest.
- Compare with second and third numbers.
- Update smallest accordingly.

Output 1 7 -5 Observation Few-shot examples helped the AI infer comparison logic correctly without using built-in functions.

Conclusion

- **Zero-shot prompting** works well for simple tasks but may lack optimization.
- **One-shot prompting** improves clarity using example.
- **Few-shot prompting** provides the best accuracy, structure, and optimized solutions.
- Providing examples improves AI understanding and output.